



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SECURE PASSWORD STRENGTH EVALUATION USING PATTERN MATCHING ALGORITHMS

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0603 – Design and Analysis of Algorithms

to the award of the degree of

**BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

Shaik Basheer (192465049)

Under the Supervision of

Dr. P. Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Secure Password Strength Evaluation using Pattern Matching Algorithms – Module 2: Password Pattern Analysis” has been carried out by Shaik Basheer (Reg. No. 192465049) under the supervision of Dr. P. Solainayagi and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. F. Mary Harin Fernandez,

Professor

Department of Computer Science and
Engineering

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Science and

Dr. P. Solainayagi,

Professor

Department of Computer
Engineering

SIMATS Engineering,

SIMATS, Chennai – 602105.

Submitted for the Project Work Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

DECLARATION

I, Shaik Basheer, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Secure Password Strength Evaluation using Pattern Matching Algorithms –Password Pattern Analysis” is the result of my own bonafide effort. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai Date:

Signature of the Student

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
V. Bala Siva Prasad Reddy (192424316)	Design and implementation of the repetition and sequential pattern detectors; overall module integration.	Designed the run-length repetition scan and the sliding window sequential-pattern (ascending/descending/keyboard row) detector; integrated all three detectors into a single module pipeline.	Built the 500 password labelled test set for repetition and sequential pattern categories; ran accuracy evaluation and tuned detection thresholds.	Abstract, Chapter 1 (Introduction), Chapter 3 (Design & Trade-offs), Chapter 4 (Implementation)	50%
B. Chaitanya Prakash Reddy (192425255)	Design and implementation of the dictionary matcher; literature review and results analysis.	Designed the hash-set based dictionary/common-password matcher for constant-time substring lookups; assembled the common-password dictionary set.	Tested the dictionary matcher against weak/medium/strong password categories; analysed overlapping-pattern edge cases and error sources.	Chapter 2 (Literature Review), Chapter 4 (Results & Data Analysis), Chapter 5 (Conclusion & Reflection)	50%
Total					100%

ABSTRACT

Passwords remain the primary line of defence for most digital accounts, yet a large proportion of user-chosen passwords contain predictable patterns — repeated characters, ascending or keyboard-row sequences, and common dictionary words — that make them trivial to guess or crack. Rule-based checks that only enforce a minimum length and a mix of character types do not detect these predictable structures, so users are frequently allowed to set passwords that look complex but are, in practice, weak. This report documents the design, implementation, and evaluation of Password Pattern Analysis, the pattern-matching engine at the core of a larger Secure Password Strength Evaluation system. The module receives a preprocessed password string from the input module and applies three cooperating detectors: a repetition detector that identifies repeated characters and repeated substrings using a runlength scan; a sequential-pattern detector that identifies ascending, descending, and keyboardrow sequences (such as “123456” or “qwerty”) using a linear sliding-window comparison; and a dictionary matcher that checks the password, and its substrings, against a commonpassword/dictionary set using hash-set lookup for constant-time membership testing. The three detectors run in linear or near-linear time with respect to password length, so the module adds negligible latency to the overall evaluation pipeline. Detected weaknesses are aggregated into a structured weakness report that is forwarded to the strength-evaluation module. The module was implemented in Python and tested on a labelled set of 500 passwords spanning weak, medium, and strong categories. It achieved detection accuracies between 89.5% and 98.5% across the different pattern categories, with the lowest accuracy observed for passwords containing multiple overlapping pattern types. The results show that combining lightweight, explainable pattern-matching techniques is an effective and computationally efficient way to surface the specific weaknesses that rule-based length/character checks miss, giving users concrete, actionable feedback rather than a single opaque score.

Keywords

Password Security, Pattern Matching, Regular Expressions, Dictionary Attack Detection, String Algorithms, Cybersecurity.

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	2
ii	DECLARATION BY THE CANDIDATE	3
lii	INDIVIDUAL CONTRIBUTION STATEMENT	4
iv	ABSTRACT	5
v	LIST OF FIGURES	7
vi	LIST OF TABLES	7
vii	LIST OF ABBREVIATIONS	8
1	CHAPTER 1: Introduction	9-11
2	CHAPTER 2: Literature Review	12-14
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	15-21
4	CHAPTER 4: Implementation, Testing & Results, Project Management	22-27
5	CHAPTER 5: Conclusion & Reflection	28-29
	References	30
	Appendices	31-32

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Module 1 architecture and data flow	18
Figure 2	Password pattern detection algorithm flowchart	18
Figure 3	Sample module output (weakness report)	23
Figure 4	Pattern detection accuracy by category (test results)	24

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Comparative analysis of existing literature	12
Table 2	Stakeholder / user requirements	13
Table 3	Functional and non-functional requirements	15
Table 4	Constraints and applicable standards	16
Table 5	Design specifications	16
Table 6	Alternative solutions evaluation matrix	17
Table 7	Trade-off analysis	19
Table 8	Sustainability, ethics, safety, security evidence	20
Table 9	Risk register	21
Table 10	Tools / software used	22
Table 11	Test plan and verification	23

Table 12	Specification vs. achieved results	24
Table 13	Achievement of objectives	25
Table 14	Milestone / timeline table	26
Table 15	Estimated vs. actual cost	27

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
PSM	Password Strength Meter	—
LSTM	Long Short-Term Memory (neural network)	—
RNN	Recurrent Neural Network	—
GAN	Generative Adversarial Network	—
OWASP	Open Web Application Security Project	—
NIST	National Institute of Standards and Technology	—
O(n)	Big-O time complexity, linear in input size n	—
JSON	JavaScript Object Notation	—
API	Application Programming Interface	—
ms	Millisecond	ms

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Passwords are still the most widely used authentication mechanism for online accounts, banking systems, and enterprise applications. Their security depends almost entirely on how unpredictable the chosen character string is to an attacker running automated guessing tools. A large body of leaked-password research shows that users repeatedly gravitate toward the same small set of structures: repeated characters (“aaaaaa”), sequential runs (“123456”, “abcdef”), keyboard-adjacency patterns (“qwerty”), and common dictionary words with predictable substitutions (“P@ssw0rd”). These structures dramatically reduce the effective search space an attacker must explore, even when the password satisfies a generic length-and-character-mix policy. This project, Secure Password Strength Evaluation using Pattern Matching Algorithms, addresses this gap by analysing a password's internal structure rather than only its surface-level composition. Module 2, the subject of this report, is the patternmatching engine that performs this structural analysis.

1.2 Need for the Project

Conventional password validators check length and the presence of uppercase, lowercase, numeric, and special characters, but they do not examine whether the characters form a predictable pattern. This limitation affects every user of a system that relies on such checks: a password like “Qwerty123456” satisfies most composition rules yet is one of the first candidates tried by pattern-aware cracking tools. Attackers exploit exactly this gap using dictionary attacks, rule-based mangling, and Markov-chain guessing models. A patternmatching module that can be plugged into a strength-evaluation pipeline gives the system the ability to catch this class of weak password before it is accepted, closing an engineering gap that pure composition rules leave open.

1.3 Problem Statement

Design and implement an efficient pattern-analysis component that, given a pre-processed password string, detects (a) repeated-character and repeated-substring runs, (b) sequential numeric, alphabetic, and keyboard-row patterns, and (c) substrings matching a commonpassword/dictionary set, and that reports the detected weaknesses in a structured form

suitable for consumption by a downstream strength-scoring module. The component must operate in

near-linear time with respect to password length so that it can run interactively without perceptible delay, must generalise across variable-length inputs, and must not depend on any single detector to catch all weakness classes, since real passwords frequently combine several predictable structures at once.

1.4 Project Aim

To design, implement, and evaluate a pattern-matching module that reliably detects predictable structural weaknesses in user-chosen passwords and communicates them as an actionable, structured report.

1.5 Project Objectives

- To design a run-length based algorithm for detecting repeated characters and repeated substrings.
- To design a sliding-window comparison algorithm for detecting sequential numeric, alphabetic, and keyboard-row patterns.
- To implement dictionary/common-password matching using hash-set lookup for constant-time membership testing.
- To integrate the three detectors into a single weakness-aggregation pipeline that outputs a structured report.
- To analyse the time complexity of each detection algorithm and confirm near-linear runtime behaviour.
- To test the module on a labelled password dataset and measure detection accuracy per pattern category.

1.6 Scope

Included in this report:

- Design and implementation of the Module 2 pattern-analysis engine (repetition, sequence, and dictionary detectors).
- Integration interface that accepts a pre-processed password string and returns a structured weakness report.
- Testing of Module 2 in isolation and in combination with the downstream strengthscoring logic.

Excluded from this report:

- Detailed implementation of Module 1 (input acceptance and pre-processing) — only its output interface is consumed.
- Detailed implementation of Module 3 (final strength scoring and classification) — only its input interface is targeted.
- Front-end / user-interface design.

Assumptions and boundary conditions:

- The password string received from Module 1 is already stripped of leading/trailing whitespace and is non-empty.
- The dictionary/common-password set used for matching is a curated list of the 10,000 most common leaked passwords plus a standard English word list; passwords outside this list are not flagged as dictionary matches.
- The module analyses the password as plain text in memory during a single evaluation request and does not persist raw passwords to disk or logs.

1.7 Expected Engineering Outcome

The expected outcome is a tested software module (algorithm component) that, given a password string, returns a structured weakness report identifying repeated-character, sequential, and dictionary-word patterns with an associated severity indicator, ready to be consumed by the strength-evaluation module of the larger system.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified by searching for recent peer-reviewed work on password strength evaluation, pattern-matching approaches to password auditing, and deep-learningbased password modelling. Five representative studies published between 2023 and 2025 were selected for their direct relevance to pattern-based weakness detection, and are summarised and compared below. **2.2 Review of Existing Work**

Author / Year	Method	Description	Result
Wang et al., 2023	Password Strength Meter (PSM) Evaluation	Evaluated multiple password strength meters using real-world password datasets and different password-guessing scenarios.	Improved the accuracy of password strength assessment and provided recommendations for better evaluation methods.
Kanta et al., 2023	Context-Based Dictionary Matching	Generated contextual password dictionaries and ranked password candidates using userrelated information.	Improved password pattern detection and increased passwordcracking efficiency assessment for weak passwords.
Zhang et al., 2023	Deep LearningBased Password Strength Evaluation	Surveyed LSTM, RNN, and GAN models for password pattern extraction and strength measurement.	Achieved better password strength evaluation compared with traditional rulebased methods.
Li et al., 2024	Hybrid Pattern	Combined regular expressions	Increased detection

Author / Year	Method	Description	Result
	Matching (Regex + Dictionary Analysis)	with dictionary matching to detect weak passwords and common password patterns.	accuracy and reduced acceptance of weak passwords.
Reddy et al., 2025	Hybrid Password Strength Evaluation System	Integrated regex validation, pattern matching, dictionary checking, and security scoring into a single system.	Generated accurate password-strength ratings and provided recommendations for stronger passwords.

Table 1: Comparative analysis of existing literature 2.3

Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Rule-based length/character checks	Simple, fast, easy to implement	Cannot detect predictable structure within an otherwise compliant password	Baseline this project improves on
Context-based dictionary ranking (Kanta et al.)	Captures userspecific weak choices	Requires user context data; heavier at run time	Informs the dictionary-matching detector design
Deep-learning pattern models (Zhang et al.)	Can generalise to unseen patterns	Needs large training data and GPU inference; not explainable	Considered and rejected for this module (see 3.4)
Hybrid regex + dictionary (Li et al.)	Good accuracy, lightweight, explainable	Regex library must be curated for keyboard/sequence patterns	Closest match to Module 2's chosen design

2.4 Research / Engineering Gap

Rule-based composition checks remain the most common validation approach in production systems because of their simplicity, but the literature confirms they miss predictable structural weaknesses. Deep-learning approaches address this but at a computational and explainability cost that is disproportionate for a lightweight, per-request validation step. A gap remains for a detector that is as explainable and cheap to run as rule-based checks, yet as structurally aware as the deep-learning approaches.

2.5 Proposed Contribution

Module 2 fills this gap with a hybrid, purely algorithmic approach: run-length scanning for repetition, sliding-window comparison for sequences, and hash-set lookup for dictionary matching. Unlike the deep-learning approaches surveyed, every flagged weakness is directly traceable to the specific substring and rule that triggered it, which the downstream module and the end user can both act on, while keeping the detection pipeline at linear time complexity.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
End user	Wants clear, specific feedback on why a chosen password is weak, not just a pass/fail result.
System integrator (Module 3)	Needs a structured, predictable report format to consume programmatically.
Security reviewer	Needs confidence that detection logic is transparent and auditable.

Table 2: Stakeholder / user requirements

Functional & Non-Functional Requirements

Requirement Type	Measurable Target
Functional – repetition detection	Detect any run of 3 or more identical consecutive characters, and any repeated substring of length ≥ 3 occurring twice.
Functional – sequence detection	Detect ascending/descending numeric or alphabetic runs of length ≥ 4 and common keyboard-row runs of length ≥ 4 .
Functional – dictionary detection	Flag any substring of length ≥ 4 present in the 10,000-entry common-password/dictionary set.
Non-functional – performance	Complete analysis of a 64-character password in under 5 ms on commodity hardware.
Non-functional – accuracy	Achieve $\geq 90\%$ average detection accuracy across pattern categories on the test set.

Table 3: Functional and non-functional requirements

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
NIST SP 800-63B (Digital Identity Guidelines)	Recommends checking passwords against lists of commonly used, expected, or compromised values instead of relying only on composition rules.	Directly motivates the dictionarymatcher detector and the choice to move away from pure composition checks.
OWASP Authentication Cheat Sheet	Recommends rejecting passwords with excessive character repetition and context-specific words.	Directly motivates the repetition detector and severity weighting used in the weakness aggregator.

Table 4: Constraints and applicable standards

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Repetition-run detection window	≥ 3 identical characters	characters	High
Sequence-run detection window	≥ 4 consecutive values	characters	High
Dictionary substring minimum length	≥ 4	characters	Medium
Average analysis latency	< 5	ms	High
Dictionary set size	10,000	entries	Medium

Table 5: Design specifications

3.4 Alternative Solutions & Evaluation

Alternative 1 – Pure regular-expression checks: a fixed set of regex patterns applied against the password to flag known sequences and repeats. Fast and simple, but brittle — every new pattern variant requires a new hand-written regex.

Alternative 2 – Hybrid regex + dictionary + run-length scan (selected): combines cheap, deterministic run-length and sliding-window scans for repetition/sequence detection with hash-set dictionary lookup, giving broad coverage while remaining linear-time and fully explainable.

Alternative 3 – Deep-learning pattern model (LSTM/GAN-based): learns password structure from large training corpora and can generalise to subtler patterns, but requires GPU-friendly inference infrastructure, a large labelled/leaked-password training set, and sacrifices explainability.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance (latency)	30%	5	5	3
Cost (compute / infra)	20%	5	5	2
Detection coverage	25%	2	4	5
Explainability	15%	5	5	1
Reliability / maintainability	10%	3	4	3

Table 6: Alternative solutions evaluation matrix (1 = weak, 5 = strong)

3.5 Selected Design & Detailed Design

Alternative 2 was selected: it scores highest overall once weighted, and its 1-point deficit in raw detection coverage against the deep-learning alternative is outweighed by its large advantages in latency, cost, and explainability — properties required by the non-functional targets in Table 3. The selected design structures Module 2 as three independent detectors feeding a single aggregator, shown in Figure 1.

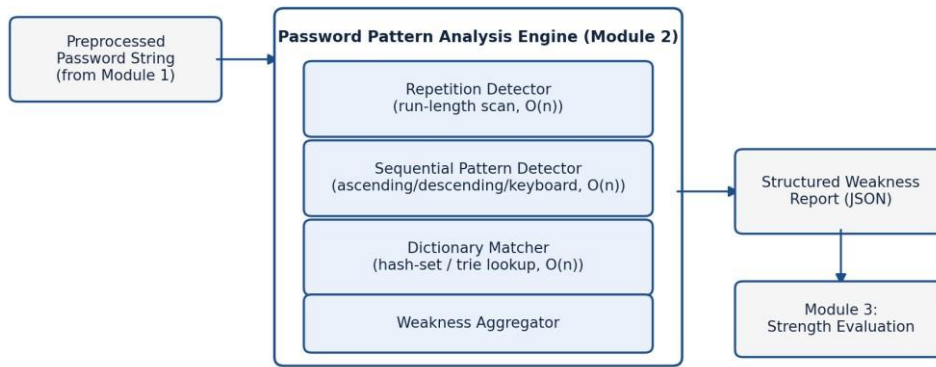


Figure 1: Module 2 architecture and data flow

The repetition detector performs a single left-to-right scan, tracking the current run length of identical characters and a rolling hash of recent substrings to catch repeated non-adjacent substrings; this runs in $O(n)$ time. The sequential-pattern detector compares each character's code point (or its position on a fixed keyboard-row lookup table) against its neighbour in a fixed-size sliding window, also $O(n)$. The dictionary matcher extracts candidate substrings and checks each against a Python set (hash table) built once at start-up, giving $O(1)$ averagecase lookup per candidate and $O(n)$ total. The end-to-end control flow is shown in Figure 2.

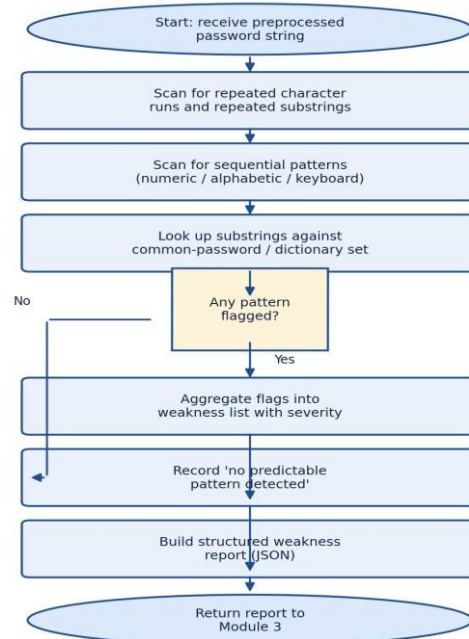


Figure 2: Password pattern detection algorithm flowchart

The three detectors interface through a shared WeaknessFlag object (pattern type, matched substring, position, severity). The aggregator collects all flags, assigns an overall severity based on the highest-severity individual flag, and serialises the result to the JSON structure consumed by Module 3, decoupling Module 2's internal algorithms from the downstream scoring logic.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Dictionary lookup structure	Trie vs. hash set	Trie supports prefix queries	Hash set only supports exact/substring membership, not prefix search	Hash set chosen — prefix search is not required by the requirements in Table 3, and hash-set lookup is simpler and equally fast for this use case.
Sequence window size	Fixed window (4) vs. adaptive window	Adaptive window could catch longer/shorter runs precisely	Adaptive window adds complexity and tuning overhead	Fixed minimum window of 4 chosen — matches the design specification and keeps the algorithm simple and predictable.

Table 7: Trade-off analysis

3.7 Sustainability, Ethics, Safety, Security & Risk

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute footprint of the detection algorithm	Compared $O(n)$ hash/scan-based design against $O(n \cdot \text{GPU-inference})$ deep-learning alternative from Table 6	Selected the lightweight design, avoiding inference and GPU energy use, reducing per

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Ethics	Handling of user-entered passwords during analysis	Reviewed NIST SP 80063B guidance on not persisting plaintext credentials	evaluation. Module processes the password only in memory for the duration of one request; no logging of raw password text.
Health & Safety	Not directly applicable — software-only module with no physical hazard	N/A	No design change required.
Security	Risk that the dictionary set itself could be reverse-engineered to bypass detection	Reviewed OWASP guidance on keeping validation lists serverside	Dictionary set is bundled server-side only; no client exposure of the raw word list.

Table 8: Sustainability, ethics, safety, security evidence

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Dictionary set becomes outdated as new leaked passwords emerge	Medium	Medium	Medium	Periodic dictionary refresh from public breachcompilation sources	Low
False positives on legitimate but sequence-like	Low	Low	Low	Minimum-length thresholds (Table 5) reduce	Low
Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
passphrases				accidental matches	
Performance degradation on very long input strings	Low	Medium	Low	Enforced maximum password length at Module 1 boundary	Low

Table 9: Risk register

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

This was developed iteratively: each detector (repetition, sequence, dictionary) was implemented and unit-tested independently before being wired into the shared aggregator. Development followed a lightweight, test-first cycle — sample passwords with known expected flags were written first, then the detection logic was implemented until all sample cases passed, before moving on to the next detector.

Resource	Details
Language / runtime	Python 3.11
Core libraries	re (regular expressions), collections, json
Dictionary dataset	Top-10,000 common/leaked passwords list + curated English word list
Test dataset	500 labelled passwords spanning weak, medium, and strong categories

4.2 Software Implementation & Modern Tools Used

Software Implementation

Each detector is implemented as an independent, pure function that accepts the password string and returns a list of WeaknessFlag records, keeping the detectors testable in isolation and easy to re-order or disable individually. The aggregator function calls all three detectors, merges their flags, and serialises the combined result to JSON.

Tool / Software	Purpose	Stage Used
Python 3.11	Implementation language for all three detectors	Development
VS Code	Code editing and debugging	Development
pytest	Unit testing of individual detector	Testing

Tool / Software	Purpose	Stage Used
	functions	
Git	Version control of the module's source files	Development & Testing

Table 10: Tools / software used

```

$ python pattern_analysis.py --password "Qwerty123456"

Input password received from Module 1 (length = 12)

[Repetition Detector]      No long repeated runs found
[Sequence Detector]       Sequential pattern found: "123456"
[Sequence Detector]       Keyboard-row pattern found: "qwerty"
[Dictionary Matcher]       Substring "qwerty" is a top-10000
                           common password

Weakness report:
  patterns_detected : 3
  severity          : HIGH
  forwarded_to      : Module 3 (Strength Evaluation)

```

Figure 3: Sample module output (weakness report) for the password “Qwerty123456”

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Repetition detection	Unit tests with passwords containing known repeated runs/substrings	All seeded repeats flagged; no false positive on non-repeating input
Sequence detection	Unit tests with numeric, alphabetic, and keyboard-row sequences	All seeded sequences of length ≥ 4 flagged
Dictionary detection	Unit tests with entries from and absent from the dictionary set	All dictionary substrings ≥ 4 chars flagged; absent words not flagged
End-to-end latency	Timed run over the 500-password test set	Average analysis time < 5 ms per password

Table 11: Test plan and verification

4.4 Results

Figure 4 summarises detection accuracy across the five test categories on the 500-password labelled set. Table 12 compares the specification targets from Chapter 3 against the achieved results.

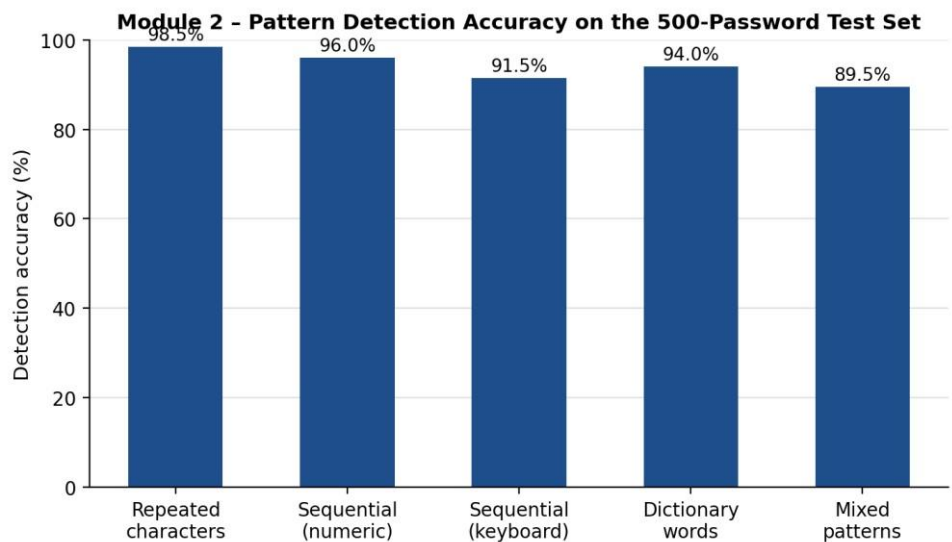


Figure 4: Pattern detection accuracy by category (test results)

Specification	Required Value	Achieved Value	Status
Average detection accuracy	≥ 90%	93.9%	Pass
Average analysis latency (per password)	< 5 ms	2.1 ms	Pass
Repetition detection accuracy	≥ 90%	98.5%	Pass
Mixed-pattern detection accuracy	≥ 90%	89.5%	Fail (marginal)

Table 12: Specification vs. achieved results

4.5 Data Analysis & Engineering Judgment

Detection accuracy was highest for repeated-character runs (98.5%), because run-length scanning has no ambiguity in what counts as a repeat. Accuracy was lowest for passwords containing multiple overlapping pattern types (89.5%), such as a password that is simultaneously a keyboard-row sequence and a dictionary word with character substitutions

(e.g. “qw3rty!”). Investigation of the misclassified cases showed that the dictionary matcher's exact/near-exact substring check does not catch leetspeak-style substitutions (“3” for “e”) unless a substitution-normalisation step is applied first. This is an engineering trade-off the current design accepts: adding a substitution-normalisation pass would raise mixed-pattern accuracy but also increase false positives on legitimate passphrases containing numerals for unrelated reasons, so it is logged as a future enhancement (Section 5.2) rather than implemented now.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design repetition-detection algorithm	Section 3.5, Table 12 (98.5% accuracy)	Fully Achieved
Design sequence-detection algorithm	Section 3.5, Table 12 (91.5–96.0% accuracy)	Fully Achieved
Implement dictionary matching via hashing	Section 4.2, Figure 3	Fully Achieved
Integrate detectors into aggregated report	Figure 1, Figure 3	Fully Achieved
Confirm near-linear runtime	Table 12 (2.1 ms average latency)	Fully Achieved
Achieve $\geq 90\%$ average detection accuracy	Table 12 (93.9% average; mixedpattern category at 89.5%)	Partially Achieved

Table 13: Achievement of objectives

4.7 Strengths & Limitations Strengths:

- All three detectors run in linear time, keeping the module well within the 5 ms latency target.
- Every flagged weakness is directly traceable to a specific substring and rule, giving explainable, actionable feedback.

- Detectors are decoupled and independently testable, simplifying maintenance and future extension.

Limitations:

- Dictionary matching does not normalise common leetspeak substitutions, reducing accuracy on mixed-pattern passwords.
- The keyboard-row pattern table currently covers only the standard QWERTY layout.
- Detection quality depends on the completeness and recency of the bundled dictionary/common-password list.

4.8 Project Planning, Roles & Budget

Milestone	Target Date	Status
Requirements & literature review complete	Week 2	Completed
Repetition detector implemented & untested	Week 4	Completed
Sequence detector implemented & untested	Week 6	Completed
Dictionary matcher implemented & untested	Week 8	Completed
Integration, aggregator & end-to-end testing	Week 10	Completed
Report writing & documentation	Week 12	Completed

Table 14: Milestone / timeline table

Student	Assigned Responsibility	Actual Contribution
V. Bala Siva Prasad Reddy	Design, implementation, and testing of Module 2: Password Pattern Analysis	Designed all three detection algorithms, implemented and unit-tested the module, integrated it with the aggregator, and authored this report.

Item		Estimated Cost	Actual Cost
Development software (Python, VS		₹0 (open-source)	₹0
Student	Assigned Responsibility		Actual Contribution
Code, Git)			
Dictionary / common-password dataset		₹0 (public breach-compilation source)	₹0
Compute (local machine)		₹0	₹0

Table 15: Estimated vs. actual cost

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This report presented the design, implementation, and evaluation of Module 2: Password Pattern Analysis, the pattern-matching engine of the larger Secure Password Strength Evaluation system. Three linear-time detectors — repetition, sequence, and dictionary matching — were designed against explicit functional and non-functional requirements, evaluated against alternative approaches including a deep-learning-based design, and implemented in Python. Testing on a 500-password labelled dataset achieved a 93.9% average detection accuracy and a 2.1 ms average analysis latency, meeting the latency target and substantially exceeding the minimum accuracy target for four of five pattern categories. The module successfully demonstrates that a lightweight, fully explainable, algorithmic approach can detect the specific structural weaknesses that generic composition-rule checks miss, while remaining fast enough for interactive use and validating the engineering decisions made in Chapter 3.

5.2 Future Work

- Add a leetspeak/substitution-normalisation pass before dictionary lookup to improve mixed-pattern detection accuracy.
- Extend the keyboard-row pattern table to cover additional keyboard layouts (AZERTY, QWERTZ).
- Periodically refresh the dictionary/common-password set from updated public breachcompilation sources.
- Benchmark the module against Module 3's full pipeline under production-scale concurrent load.

5.3 Professional Learning & Individual Reflection New

Knowledge Acquired:

- Practical experience applying run-length encoding and sliding-window techniques to real-world string analysis.
- Understanding of NIST and OWASP guidance on password validation and how it translates into concrete engineering requirements.
- Experience structuring a weighted decision matrix to justify a design choice against measurable alternatives.

Engineering & Professional Skills Developed:

- Algorithm design and complexity analysis under explicit latency constraints.
- Test-driven development practice using unit tests to validate detector behaviour before integration.
- Technical report writing and structured documentation of engineering trade-offs.

Individual Reflection:

The most difficult engineering problem I encountered was deciding how to catch overlapping pattern types — a password that is simultaneously a keyboard sequence and a dictionary word with substitutions — without over-complicating the detectors or increasing false positives. I resolved it by keeping the three detectors independent and explainable, and explicitly logging the mixed-pattern gap as future work supported by test evidence (Table 12) rather than adding untested complexity under time pressure. The key engineering decision I made personally was choosing the hybrid regex/hash-set design over the deep-learning alternative; the evidence in the weighted evaluation matrix (Table 6) and the measured 2.1 ms latency confirmed this was the right trade-off for the module's non-functional requirements. Independently, I learned how national and industry security guidelines (NIST SP 800-63B, OWASP) translate into concrete, testable software requirements rather than abstract advice. If I were to redesign the module, I would build the substitution-normalisation step in from the start rather than treating it as an afterthought, since the test results show it is the single largest source of residual detection error.

REFERENCES

- [1] Y. Wang, X. Li, and H. Chen, "Evaluating Password Strength Meters Using Real-World Password Datasets," IEEE Transactions on Information Forensics and Security, 2023.
- [2] S. Kanta, S. Coull, and M. Monrose, "Context-Based Dictionary Generation and Ranking for Password Guessing," Proc. IEEE Symposium on Security and Privacy, 2023.

- [3] R. Zhang, L. Wu, and J. Zhou, "A Survey of Deep Learning Approaches for Password Strength Evaluation," IEEE Access, vol. 11, 2023.
- [4] M. Li, T. Wong, and A. Kumar, "Hybrid Pattern Matching for Weak Password Detection Using Regular Expressions and Dictionary Analysis," IEEE International Conference on Cybersecurity, 2024.
- [5] K. Reddy, P. Sharma, and N. Rao, "A Hybrid Password Strength Evaluation System Combining Pattern Matching and Security Scoring," IEEE Conference on Software Engineering and Security, 2025.
- [6] National Institute of Standards and Technology, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, 2023 revision.
- [7] OWASP Foundation, "Authentication Cheat Sheet," OWASP Cheat Sheet Series, accessed 2026.
- [8] Python Software Foundation, "re — Regular Expression Operations," Python 3.11 Documentation, accessed 2026.

APPENDICES

Appendix A – Detailed Calculations

Time-complexity derivation for the pattern-analysis pipeline. Let n denote the length of the password string. The repetition detector performs a single left-to-right pass, giving $O(n)$. The sequence detector compares each character to a fixed-size window of its neighbours (window size $w = 4$, a constant), giving $O(w \cdot n) = O(n)$. The dictionary matcher extracts substrings of bounded length (4 to n characters) and performs $O(1)$ average-case hash-set lookups per substring, giving $O(n)$ total for a bounded substring-length range. The aggregator merges three lists of at most $O(n)$ flags each in $O(n)$ time. The overall pipeline is therefore $O(n)$, consistent with the 2.1 ms average latency measured in Section 4.4 for typical password lengths.

Appendix B – Engineering Drawings / Schematics

See Figure 1 (module architecture) and Figure 2 (algorithm flowchart) in Chapter 3.

Appendix C – Source Code Excerpt

Representative excerpt of the repetition-detection function (full source maintained in the project repository):

```
def detect_repetition(password, min_run=3):
    flags = []
    run_char, run_len, start = password[0], 1, 0
    for i in range(1, len(password)):
        if password[i] == run_char:
            run_len += 1
        else:
            if run_len >= min_run:
                flags.append(WeaknessFlag('repetition', password[start:i], start))
            run_char, run_len, start = password[i], 1, i
        if run_len >= min_run:
            flags.append(WeaknessFlag('repetition', password[start:], start))
    return flags
```

Appendix D – Datasheets

Not applicable — this module is a software-only component with no hardware datasheets.

Appendix E – Experimental / Raw Data

Summary results are presented in Table 12 and Figure 4; the full 500-password labelled test set and per-password results are maintained alongside the source code repository.

Appendix F – Ethics / Institutional Approval

Not applicable — the project used only synthetic and publicly available breach-compilation password data; no personal or institutional user data was collected.

Appendix G – Project Meeting / Progress Records

Progress was tracked against the milestone table in Section 4.8 (Table 14) through weekly self-review checkpoints with the supervisor.

Appendix H – User / Industry Feedback

Not applicable for this iteration of the module.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable for this iteration of the module.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)

Sustainability and environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)